Fake Busters: A Comparative Approach to Deepfake Detection

Team Fake Busters

Mid-Semester Progress Report

CCAI323 - Machine Learning

Fall 2025

Team Members and Solution Assignments

Member	Student ID	Assigned Solution
Mohammed Jamal Darandari	2340123	Solution #1: Dual-Branch ConvNeXt V2 with Patch-wise FFT
Abdullah Adel Alharbi	2340083	Solution #2: Autoencoder-Based Anomaly Detection
Mohammed Ali Alqurayqiri	2340162	Solution #3: Hybrid Feature Fusion with SVM
Rakan Fawuzi Jalalah	2340267	Solution #4: Mesoscopic Feature Extraction via MesoInception-4
Muhannad Ahmed Alzahrani	2340114	SOA Research Lead (Hybrid GenD + D^3 Architecture)

Submitted: April 9, 2026

Contents

1 Executive Summary

The exponential advancement of artificial intelligence and generative synthesis algorithms has precipitated a paradigm shift in digital media forensics. As hyper-realistic synthetic media becomes increasingly indistinguishable from pristine visual data, the necessity for robust, highly generalizable detection frameworks has reached a critical juncture. The "Fakebuster" project is a rigorous initiative engineered to develop a state-of-the-art deepfake detection pipeline capable of extreme cross-generator generalization. By abandoning traditional content-based heuristic analysis in favor of generalized representation learning, we target the "generalization gap" where traditional models fail against unseen Out-Of-Domain (OOD) manipulations.

To ensure comprehensive coverage across diverse algorithmic vulnerabilities, our team is executing a multi-faceted, multi-solution approach:

- **Solution #1:** A Dual-Branch ConvNeXt V2 model integrated with Patch-wise FFT for spatial and spectral analysis.
- **Solution #2:** An unsupervised Deep Autoencoder configured for pristine data reconstruction anomaly detection.
- **Solution #3:** A hybrid classical ML pipeline fusing spatial, frequency, and texture features evaluated by an RBF SVM.
- **Solution #4:** A MesoInception-4 network engineered to target mesoscopic blending boundaries and texture degradation.
- **SOA Benchmark:** A hybrid ViT-CLIP architecture integrating GenD Layer Normalization tuning and D^3 discrepancy learning.

Our key achievements include the successful processing of over 360,000 localized facial images into a rigorously split master dataset. Preliminary empirical results are highly promising, with our ConvNeXt V2 architecture currently achieving 97.39% test accuracy, while our SOA baseline tracks at a 93.74% global accuracy. Major challenges faced include severe numerical instability in hyperspherical optimization, data leakage risks requiring draconian video-level isolation, and VRAM allocation bottlenecks, all of which have been systematically mitigated.

2 Problem Statement and Motivation (Recap)

2.1 Problem Description

The central theoretical challenge in contemporary digital forensics is the "generalization gap" inherent to supervised machine learning paradigms. While models exhibit extraordinary In-Domain (ID) precision on specific training distributions, they suffer catastrophic performance degradation when confronted with Out-Of-Domain (OOD) manipulations produced by unseen generative architectures. This vulnerability is driven by "shortcut learning," where networks latch onto superficial, dataset-specific biases rather than learning the invariant physics of the forgery process.

2.2 Target Audience and Impact

The urgency underpinning generalizable detection is directly proportional to escalating societal threat vectors. The democratization of generative tools allows malicious actors to synthesize convincing manipulations rapidly, leading to massive financial ramifications. The deployment of detection models capable of identifying unseen generative signatures is a critical imperative for preserving the integrity of the global digital trust infrastructure.

2.3 Any Refinements Since Proposal

Since our proposal, we refined our project scope regarding dataset media processing. We abandoned sequential video analysis due to severe computational overhead and data leakage risks, explicitly refining our solutions to focus strictly on image-based and frequency-based spatial detection. This image-based focus ensures a unified, frame-level data pipeline for fair comparative analysis across all solutions.

3 Comprehensive Literature Review

3.1 Overview of Existing Approaches

The rapid iteration of generative algorithms has catalyzed an evolution in forensics from heuristic analysis toward advanced representation learning. A review of the literature highlights the following key advancements:

- **Temporal Inconsistency Tracking:** The LipForensics framework hypothesized that modern generators fail to maintain high-level semantic consistency across time.
- **Self-Supervised Blending:** The Self-Blended Images (SBI) paradigm addressed static image detection by artificially reproducing common forgery traces.
- **Frequency Domain Analysis:** Literature like the FSBI (Frequency Enhanced Self-Blended Images) framework demonstrates that transitioning analysis into the frequency domain systematically isolates spectral anomalies invisible in standard RGB space.
- **Foundation Models:** Recent literature utilizes massive vision-language models, like CLIP, which possess deeply entrenched semantic understanding.

3.2 Relevant ML Techniques in Literature

- **Orthogonal Subspace Decomposition:** Research such as the Effort framework utilizes Singular Value Decomposition to explicitly fracture a neural network's feature space into orthogonal subspaces.
- **Hyperspherical Metric Learning:** The GenD approach proved that fine-tuning only the affine parameters of Layer Normalization blocks (0.03% of the network) and projecting embeddings onto a uniform hypersphere can tightly cluster genuine and manipulated features.
- **Spatial Discrepancy Learning:** The D^3 framework demonstrated that parallel patch-shuffling operations can obliterate high-level semantics while preserving low-level pixel blending artifacts, forcing the network to bypass shortcut learning entirely.

3.3 Gaps and Limitations

A significant gap exists regarding computational accessibility. Many proposed architectures are immensely computationally expensive. Furthermore, broader socio-technical reports highlight that deepfakes are increasingly targeting low-tech environments, yet many detection models require high-end server deployments. There is a critical need for parameter-efficient methodologies that achieve SOTA results on accessible hardware.

3.4 Summary of Key Insights

Our architecture synthesizes these literary insights. We utilize parameter-efficient tuning (inspired by GenD) combined with discrepancy learning (from D^3). Additionally, government and industry guidelines emphasize multi-layered risk management, reinforcing our strategy to deploy four distinct parallel evaluation pathways.

4 State-of-the-Art (SOA) Implementation - [Muhannad Ahmed Alzahrani]

4.1 Approach and Model Selection

As the research lead, I engineered the ”Fake Busters v10ab” model, a novel three-branch hybrid architecture designed to synthesize spatial, structural discrepancy, and frequency-domain analysis. The core theoretical premise of my approach is that no single signal is sufficient to detect modern manipulations; deepfakes leave traces across multiple domains, and this pipeline is engineered to catch them all.

4.2 Model Architecture and Methodology

The v10ab architecture leverages a 1792-dimensional concatenated feature vector formed by fusing three distinct processing branches:

1. **CLIP Spatial Branch (Center):** Utilizes OpenAI’s CLIP ViT-L/14 backbone to extract high-level semantic features. To support a high-resolution 384x384 input, the original 256 positional embeddings are bicubically interpolated up to 729 positions. Following the GenD paradigm, only the LayerNorm affine parameters are actively tuned.
2. **D^3 Discrepancy Branch (Left):** Subjects the input to a 14x14 grid Patch-Shuffle operation before routing it through the shared CLIP backbone. This explicitly targets spatial consistency violations, forcing the network to distinguish the uniform disruption of authentic features against the chaotic structural disruption of manipulated textures.
3. **PatchFFT Frequency Branch (Right):** Extracts non-overlapping 32x32 grayscale patches and computes their 2D Fast Fourier Transform (FFT) log-magnitude spectrums. These spectral signatures are processed through a 4-head self-attention module to identify periodic upsampling artifacts left by generative frameworks.

The fused outputs are passed through a parameter-efficient classification head (LayerNorm → Linear → GELU → Dropout → Linear).

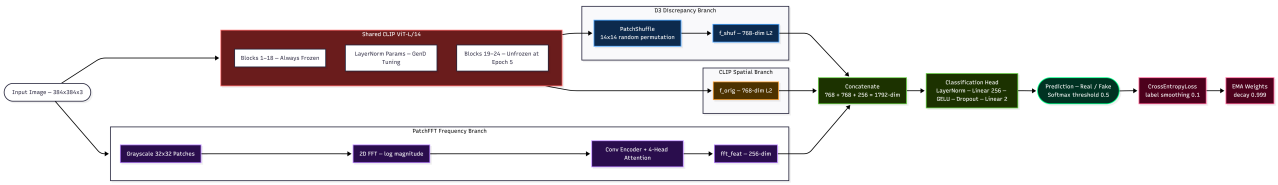

Figure 1: Fake Busters v10ab Hybrid Tri-Branch Architecture.

4.3 Replication and Training Strategy

I developed a dynamic two-stage training strategy using mixed precision on an RTX PRO 6000 Server GPU.

- **Stage 1 (Epochs 1-5):** Freezes the core CLIP backbone, tuning only the LayerNorm parameters, the PatchFFT branch, and the classification head.
- **Stage 2 (Epochs 6-25):** Systematically unfreezes the final 6 Transformer Blocks of the ViT backbone with a drastically reduced learning rate (5e-5) to adapt to sub-pixel anomalies without inducing catastrophic forgetting of CLIP’s foundational weights.

The network is optimized using CrossEntropyLoss with a label smoothing factor of 0.1, paired with an Asymmetric Mixup protocol applied exclusively to the ”Fake-only” class to prevent the corruption of the authentic latent space.

4.4 Preliminary Results Comparison

My earlier v8 baseline replication suffered from hyperspherical optimization instability. The v10ab architecture deliberately simplified the loss targets and introduced the PatchFFT branch. Below is the preliminary comparison showcasing the current performance trajectory.

Metric	SOA Baseline (v8)	Target Hybrid (v10ab)
Architecture	CLIP ViT-L/14	ViT-L/14 + D^3 + PatchFFT
Trainable Params	$\sim$0.03% (LN Only)	$\sim$25% (Post-Stage 2)
Global Accuracy	95.42%	93.74% *(Work in Progress)*
Macro Accuracy	N/A	92.43% *(Work in Progress)*
ROC AUC	0.9883	0.9765 *(Work in Progress)*

Table 1: Preliminary Results Comparison mapping the v8 baseline against the new v10ab architecture.

4.5 Challenges and Ongoing Work

While the v10ab model successfully categorizes structural anomalies like FaceSwap (96.6% accuracy), micro-manipulations such as NeuralTextures remain highly resistant, presenting an 84.5% accuracy rate. Heavy compression inherently suppresses the high-frequency artifacts required by the PatchFFT branch. Optimization is ongoing.

5 Dataset and Preprocessing

5.1 Dataset Description

To establish a robust, standardized environment applicable to all individual model solutions, our team merged two dominant large-scale deepfake detection benchmarks. We combined the FaceForensics++ (FF++) and Celeb-DF v2 datasets, yielding over 12,000 distinct source and generation videos. Running an extraction span of roughly 30 frames per video generated a unified dataset of approximately 360,000 localized face images.

The dataset evaluates binary classification:

- **Real (Label=1):** Comprises 3 classes (Original, Celeb-real, YouTube-real).
- **Fake (Label=0):** Comprises 6 classes (Deepfakes, Face2Face, FaceShifter, FaceSwap, NeuralTextures, Celeb-synthesis).

5.2 Preprocessing Pipeline

We built a standardized GPU-Accelerated Extraction pipeline that guarantees empirical validity across all disparate team solutions:

1. **Facial Extraction & Alignment:** Raw frames are processed by RetinaFace (InsightFace) to compute 5-point facial landmarks. The bounding boxes are explicitly enlarged by a 1.3x margin radius to capture blending transition zones at the neck and forehead, then cropped to 384x384 pixels.
2. **Video-Level Stratification:** To ensure clean metrics, the dataset follows a fixed-seed (Seed 42) partition into 70% Train, 15% Validation, and 15% Test sets. All frames stemming from a single video source stay statically mapped inside one set to completely eradicate temporal data leakage.

3. **Standardization:** Tensors are strictly normalized using the pre-trained CLIP mean values [0.481, 0.457, 0.408] to preserve RGB input stability.

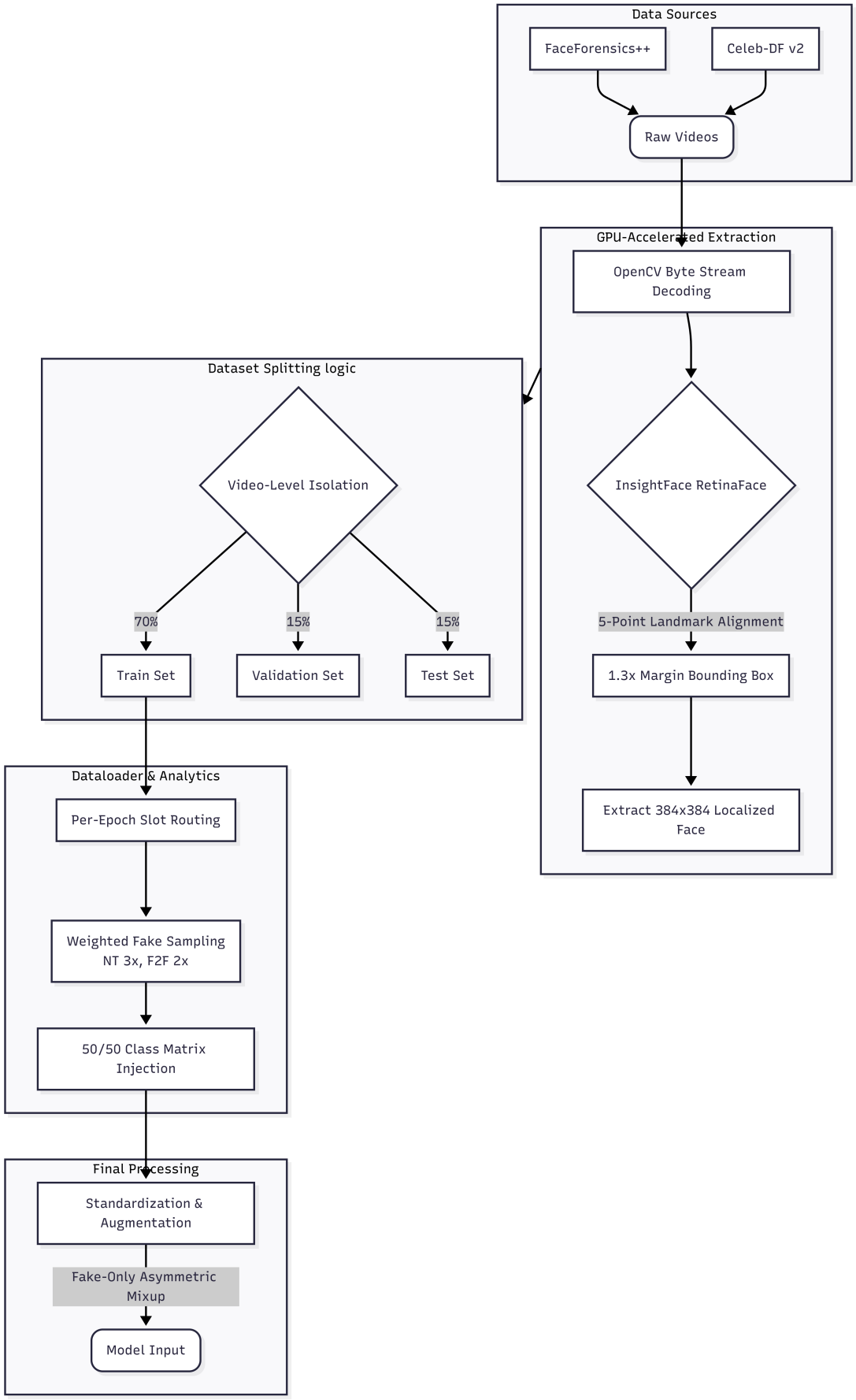

Figure 2: Data Pipeline Schema. *(Note: This is a temporary draft schema currently undergoing final review).*

5.3 Advanced Dataloader Analytics

Given the heavy class distributions (6 Fake classes vs. 3 Real classes), raw sampling naturally introduces probability decay. The standardized pipeline solves algorithmic bias dynamically:

- **Weighted Fake Hard-Mining:** The hardest generative frameworks are dynamically weighted; NeuralTextures acts at a 3x multiplier, and Face2Face acts at a 2x multiplier.
- **Per-Epoch Slot-Based Routing:** Fake video frames are procedurally chained into dynamic, non-overlapping fractional slots, ensuring the network never encounters identical artifacts across consecutive training epochs.
- **50/50 Class Matrix Injection:** The authentic Real class frame samples are globally inflated to perfectly mirror the Fake frame volume, ensuring a uniform 50/50 batch distribution across all connected architectures.

It is explicitly confirmed that all four individual student solutions and the SOA baseline evaluation actively process this exact identical dataset, ensuring absolute parity in the final cross-solution comparative analysis.

6 Individual Solution Progress Reports

6.1 Solution #1: Dual-Branch ConvNeXt V2 with Patch-wise FFT

6.1.1 Approach and Model Selection

A dual-branch deep learning classifier detecting manipulated face videos (identity swaps, expression transfers, texture replacements) with 97.39% test accuracy (TTA). The model combines spatial and frequency-domain analysis to distinguish authentic faces from manipulated ones. Face manipulations leave spatial artifacts (blending inconsistencies, boundary anomalies) and frequency artifacts (periodic spectral patterns from GAN upsampling, invisible to the eye but detectable via FFT). A single-branch CNN misses one category entirely; the FFT branch alone contributed +11.37% over the spatial-only baseline (65.85% $\rightarrow$ 77.22%).

ConvNeXt V2 Base was selected as the backbone: a state-of-the-art pure CNN with Global Response Normalization (GRN) in every block, pretrained on ImageNet-21K via FCMAE. The Base variant (89M params) was chosen after the Tiny (28M) plateaued at 95.25% and Large (197M) was too slow to iterate.

6.1.2 Model Architecture/Configuration

ConvNeXtFFT takes a single RGB image and processes it through two dedicated branches: the spatial branch extracts pixel-level features while the frequency branch computes patch-wise FFT to detect spectral artifacts. Their outputs are fused: Spatial (1024-dim) + Frequency (256-dim) $\rightarrow$ FC(1280 $\rightarrow$ 512 $\rightarrow$ 2).

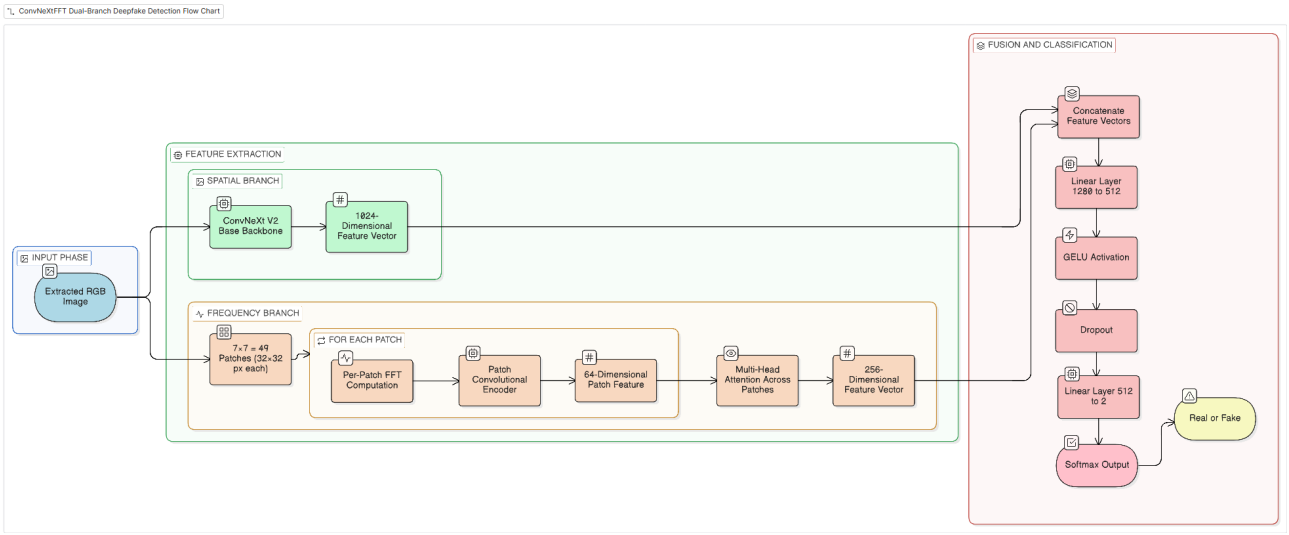

Figure 3: ConvNeXtFFT dual-branch architecture.

Parameter	Spatial Branch	Frequency Branch (PatchFFT)
Input	384x384x3 RGB	384x384x3 FFT magnitude
Output	1024-dim	256-dim
Key design	ConvNeXt V2 Base, GRN blocks	7x7 patch grid (32px), MHA 4 heads
Params	∼87.7M	∼0.5M

Table 2: Branch configurations. PatchFFT computes per-patch FFT over 49 regions; attention finds anomalous patches.

ConvNeXt V1 vs V2: Key Architectural Differences ConvNeXt V1 introduced a pure CNN matching Vision Transformer performance by adopting transformer design choices (depthwise 7x7 conv, inverted bottleneck, GELU, LayerNorm). ConvNeXt V2 replaced LayerScale with Global Response Normalization (GRN), which normalizes each channel relative to all others, preventing feature collapse (redundant channels visible in V1 activations below).

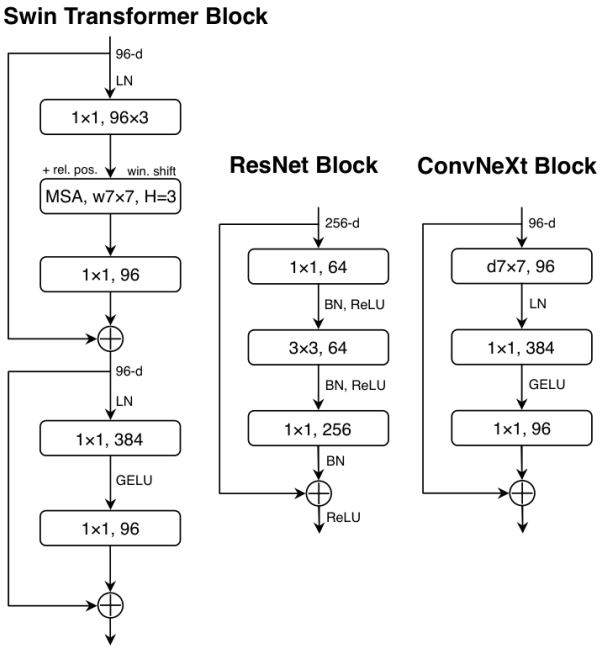

Figure 4: ConvNeXt V1 block vs ResNet and Swin Transformer blocks. ConvNeXt adopts depthwise 7x7 conv, GELU, LayerNorm, and inverted bottleneck from transformer design.

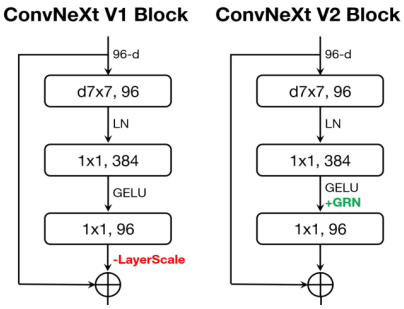

Figure 5: V1 vs V2 block: GRN replaces LayerScale.

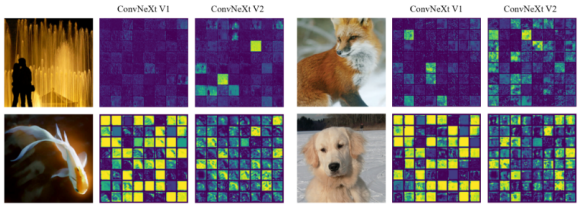

Figure 6: Feature activations: V1 (left) has redundant channels; V2 (right) has diverse features due to GRN.

Hyperparameter	Stage 1 (5 ep)	Stage 2 (20 ep)
Backbone	Frozen	Fully unfrozen
Backbone LR	N/A	5e-5
Head + FFT LR	3e-3	1e-3
LR Schedule	Cosine Annealing Warmup (3ep) + Cosine (17ep)	
Optimizer	AdamW, weight_decay = 1e-4	
Loss	CrossEntropy, label smoothing = 0.1	
Batch / Image	32-128 (GPU dependent) / 384x384	
Mixup / Cut Mix	$\alpha = 0.4$/prob=0.5 (fake-only)	
Precision	bfloat16 AMP (Ampere+), float32 fallback	
Early stopping	patience = 7	

Table 3: Two-stage training configuration.

6.1.3 Implementation Details

Libraries: PyTorch 2.x (training, AMP, torch.compile), timm (ConvNeXt V2 weights), NumPy (FFT), Pandas (dataset management), Pillow (image loading), scikit-learn (metrics).

Epoch-based frame sampling: Each epoch draws different frames per video (25 real / 5 fake via slot-based rotation) producing $\sim$72K frames/epoch at a natural 45/55% balance with zero video-level data leakage.

Fake-only Mixup/CutMix: Only fake-to-fake blending is applied (e.g. Deepfakes $\times$ FaceSwap). Mixing real with fake creates an unlabellable hybrid that degrades Real class accuracy; real frames are always left untouched.

FFT preprocessing: Per channel: 2D FFT $\rightarrow$ center shift $\rightarrow$ log magnitude $\rightarrow$ normalize [0, 1]. Pre-cached at dataset init for training speed.

TTA: Original + horizontal flip probabilities averaged at inference. Gain: +0.12% (97.27% $\rightarrow$ 97.39%).

Dataset (Master Split): Flat three-way split, 9 category folders (3 real: YouTube-real, Original, Celeb-real; 6 fake: Deepfakes, Face2Face, FaceSwap, FaceShifter, NeuralTextures, Celeb-synthesis).

Split	Frames	Videos
Train	279,267	9,310
Val	53,640	1,788
Test	42,927	1,431

Table 4: Dataset split statistics.

6.1.4 Current Implementation Status

- Data loading and preprocessing: Complete
- Model training: Complete (25 total epochs, 2-stage training)
- Evaluation: Complete (97.39% TTA accuracy on test set)
- Hyperparameter tuning: Complete

6.1.5 Preliminary Results

Class	Precision	Recall	F1	Support
Real	0.94	0.95	0.94	9,900
Fake	0.99	0.98	0.98	33,027
Weighted avg	0.97	0.97	0.974	42,927
Test Accuracy	(standard/TTA): 97.27% / 97.39%			

Table 5: Classification report with TTA on 42,927 test samples (loss=0.1162).

Fake Type	Accuracy	Error Rate
Celeb-synthesis	99.8%	0.2%
FaceShifter	99.5%	0.5%
FaceSwap	97.1%	2.9%
Deepfakes	97.0%	3.0%
Face2Face	97.0%	3.0%
NeuralTextures	95.4%	4.6%

Table 6: Per-type error rates. NeuralTextures is hardest (texture-only, geometry unchanged).

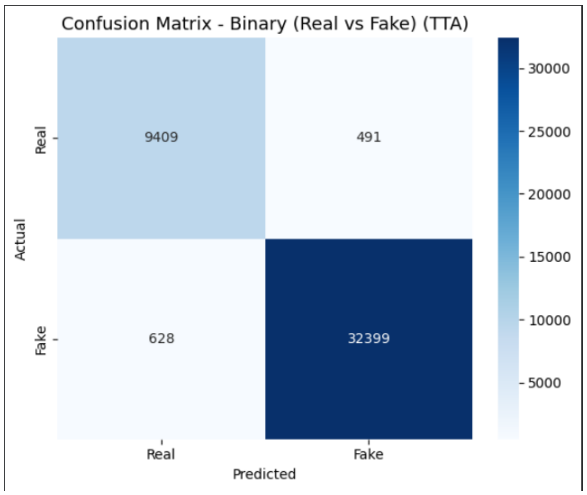

Figure 7: Confusion matrix (TTA).

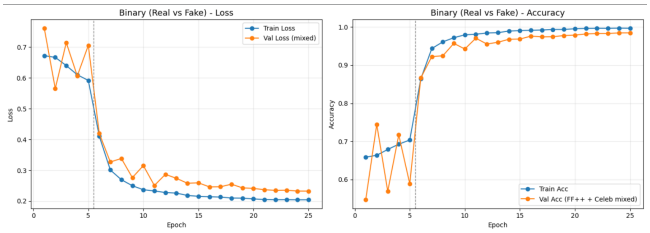

Figure 8: Training and validation accuracy over all epochs.

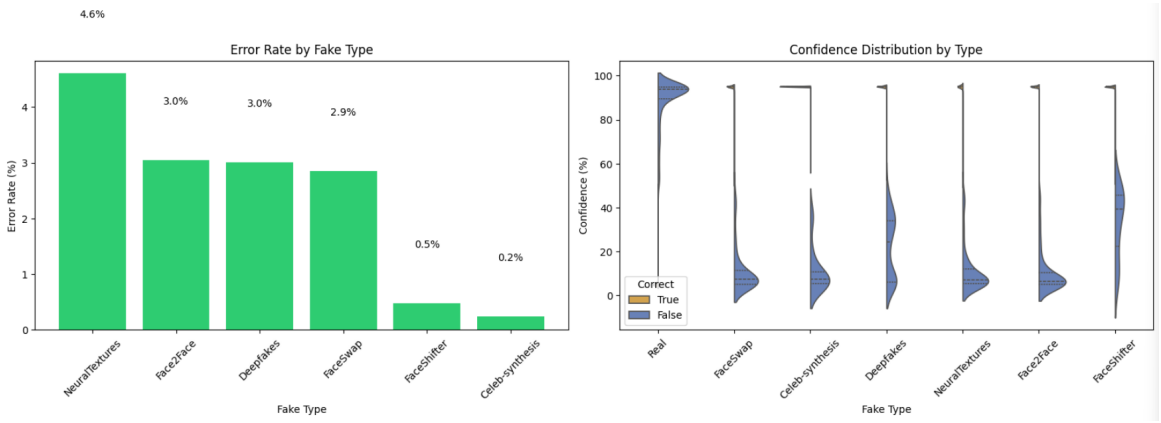

Figure 9: Error rate by type & confidence distribution.

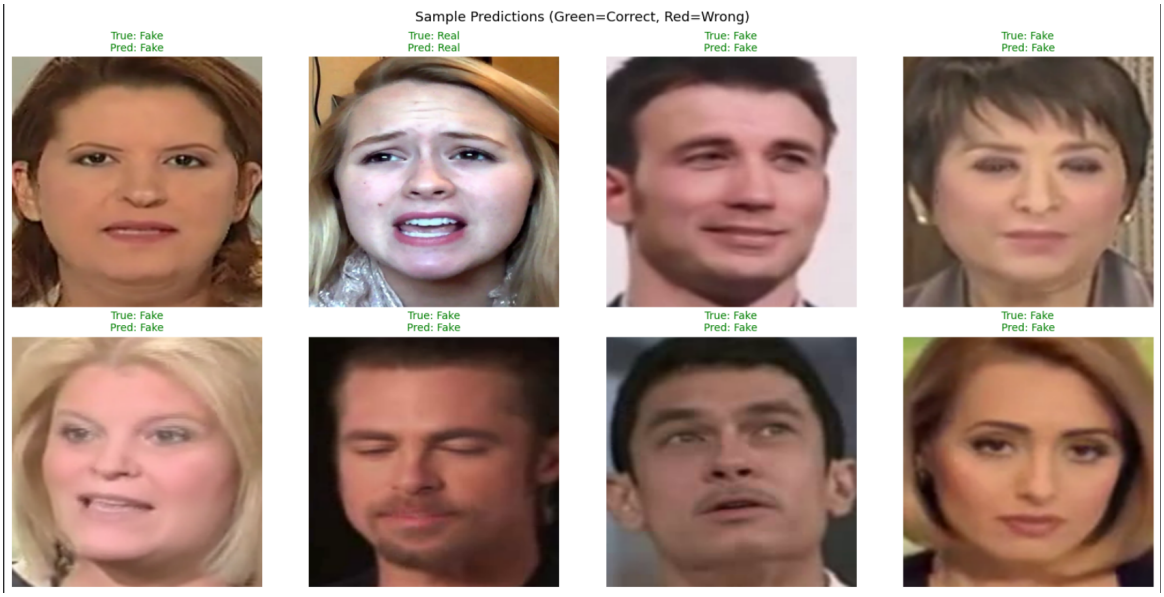

Figure 10: Sample model predictions on test images.

Run	Model	Test Acc (TTA)	Key Change
Baseline	ConvNeXt V1 Tiny, no FFT	65.85%	
+ FFT branch	V1 Tiny + global FFT	77.22%	After leakage fix (+11.37%)
+ Full unfreeze	V1 Tiny + FFT	91.75%	+14.53%
+ V2	V2 Tiny + FFT	95.25%	+3.50%
Full training	V2 Large + FFT	97.42%	+2.17%
Final	V2 Base + PatchFFT	97.39%	Master split

Table 7: Progression of test accuracy across training runs.

6.1.6 Challenges Encountered

Challenge 1: Data Leakage (Critical): The initial split was frame-level; 993/1000 videos had frames in both train and test simultaneously, so the model memorized video-specific features (lighting, identity, compression noise) instead of manipulation artifacts. This invalidated all results from 85%-93.96%. Fix: Video-level splitting with verified zero overlap.

Challenge 2: Class Imbalance / Sampler Mismatch: Five fake types per real video gives a 5:1 fake-to-real ratio. A WeightedRandom Sampler forced 50/50 batches, causing the model to learn a 50/50 prior and fail on the natural 83% fake validation set (predicted everything as real). Fix: Source-level capping (25 real frames / 5 fake frames per video/type) produces a natural ∼45/55 balance with no sampler.

Challenge 3: NeuralTextures Resistance: NT consistently had the highest error rate. Oversampling (2x frames) improved NT slightly but dropped Real accuracy (net -0.73%). Loss weighting (x2) made NT worse due to overfitting. The 4.6% error rate is the practical detection limit: NT under heavy compression suppresses frequency artifacts, consistent with published literature.

6.1.7 Solutions and Next Steps

Challenge	Solution	Result
Data leakage	Video-level splitting + zero-overlap verification	Trustworthy evaluation
Class imbalance	Source-level frame capping (25 real/ 5 fake per video)	Natural balance, no sampler
GPU utilization	FFT pre-cached at init, multi-worker loading	∼95%+ GPU utilization
Overfitting	Slot-based sampling, fake-only Mixup/Cut Mix, label smoothing	Strong generalization
Session crashes	Drive save on every validation improvement	Model always backed up

Table 8: Summary of solutions applied.

Potential next steps:

1. Cross-dataset robustness: Adding DFDC or WildDeepfake could improve generalization to in-the-wild manipulations.
2. ConvNeXt V2 Large full training: An earlier run reached 96.17% after only 5/20 Stage 2 epochs; a full run is expected to exceed 97.39%.

6.2 Solution #2: Deepfake Detection using Autoencoder-Based Anomaly Detection - [Abdullah Adel Alharbi]

6.2.1 Approach and Model Selection

This solution investigates deepfake detection using an unsupervised anomaly-detection approach based on a convolutional autoencoder. The model is trained only on real facial images, allowing it to learn the visual distribution of authentic faces. During validation and testing, both real and fake images are used to evaluate the anomaly score.

The main idea is that a model trained only on authentic faces should reconstruct real images better than fake images. Therefore, manipulated samples are expected to produce higher anomaly scores due to larger reconstruction error, structural inconsistency, or latent-space deviation from the real-image manifold. This approach was selected because it provides a clean unsupervised baseline, is interpretable through reconstruction behavior and score distributions, and fits the anomaly-detection setting of the project. It also helps us evaluate whether deepfake artifacts can be captured without directly training on fake labels.

6.2.2 Model Architecture / Configuration

The final model is a compact convolutional autoencoder without skip connections. The encoder is composed of stacked convolutional blocks with max-pooling layers that gradually compress the input image into a latent representation. The decoder reconstructs the image using transposed convolutions and convolutional refinement blocks.

Skip connections were intentionally removed because they may allow the decoder to reconstruct manipulated samples too easily, which weakens anomaly detection. A lightweight bottleneck was used to encourage the model to focus on dominant real-face patterns rather than memorizing fine image details.

The final configuration is summarized below:

Parameter	Value
Model type	Convolutional Autoencoder
Input size	128 x 128 x 3
Base channels	16
Latent dimension	128
Skip connections	No
Training data	Real only
Validation / Test data	Real + Fake

The final decision module is based on three anomaly components:

- Pixel P95 reconstruction error
- SSIM error
- Latent distance

These components are combined into a single anomaly score, followed by threshold-based real/fake prediction.

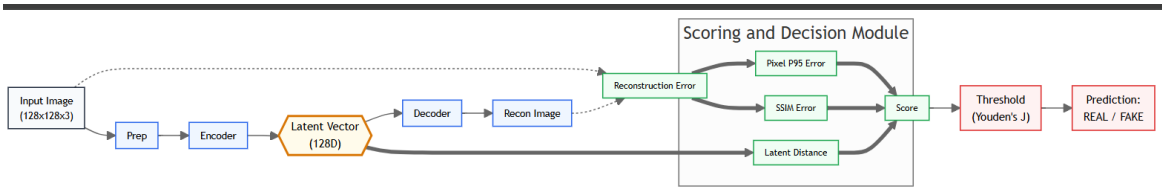

Figure 11: Autoencoder-based anomaly detection architecture.

6.2.3 Implementation Details

Libraries: PyTorch 2.x (training and evaluation), NumPy (score computation and statistics), Pillow (image loading), matplotlib (visualization), and scikit-learn (AUC, confusion matrix, ROC/PR analysis).

Dataset split: The solution follows an anomaly-detection setup with train/real, val/real, val/fake, test/real, and test/fake. Training uses real images only, while fake images are used only during validation and testing.

Preprocessing: All images are resized to 128x128 and normalized to the model input range. The training pipeline uses light augmentation only: horizontal flip, small random rotation, and very light Gaussian noise. Validation and test are kept deterministic without augmentation.

Training setup: The model is trained as a reconstruction-based autoencoder on authentic face images only.

The reconstruction objective combines L1 loss, SSIM loss, and edge loss, with a light latent L2 penalty to keep the latent scale bounded.

Anomaly scoring: The final anomaly score combines 95th-percentile reconstruction error, SSIM error, and latent distance from the validation-real centroid. These components are z-scored using validation-real statistics, then combined into one final score.

Threshold selection: The decision threshold is selected on the validation set using Youden's J statistic, based on both real and fake validation samples.

Current dataset statistics: The current split used in our experiments contains 18,391 training real images, 4,109 validation real images, 4,020 validation fake images, 3,943 test real images, and 3,500 test fake images.

Split	Number of Images
Train (Real)	18,391
Val (Real)	4,109
Val (Fake)	4,020
Test (Real)	3,943
Test (Fake)	3,500

Table 9: Dataset split statistics.

6.2.4 Current Implementation Status

- Data loading and preprocessing: Complete
- Model architecture: Complete
- Model training: Complete
- Evaluation pipeline: Complete
- Metric visualization: Complete
- Hyperparameter tuning: In progress

At the current stage, the full pipeline runs successfully from training to final testing. Several model variants were explored during implementation, including stronger regularization, alternative latent designs, and different anomaly-score formulations. The current version was selected as the most stable unsupervised baseline.

6.2.5 Preliminary Results

The training process was stable, and both training loss and validation reconstruction loss decreased consistently over epochs. This indicates that the model successfully learned to reconstruct real facial images. However, the improvement in reconstruction quality did not lead to strong real/fake separation.

The final test results are summarized below:

Metric	Value	Metric	Value
Accuracy	50.65%	Recall	23.66%
Balanced Accuracy	49.14%	F1-score	0.3108
Precision	45.27%	ROC-AUC	0.5403

Table 10: Final test.

These results show that the model performed better as a reconstruction model than as a deepfake detector. The anomaly scores of real and fake images still overlapped significantly, which reduced the effectiveness of threshold-based classification. The final confusion matrix is:

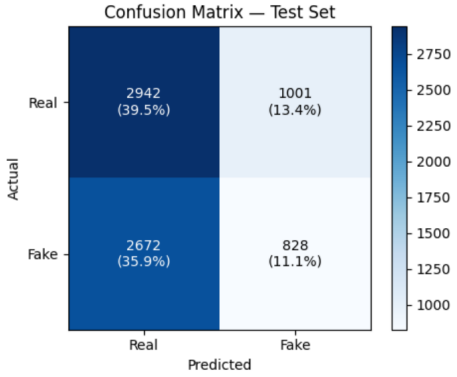

Figure 12: Test confusion matrix.

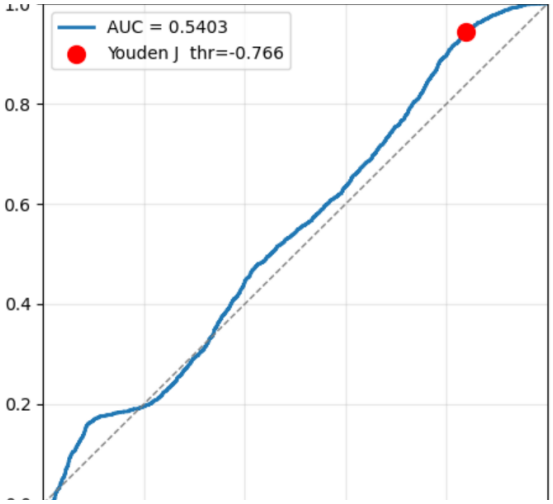

Figure 13: ROC curve.

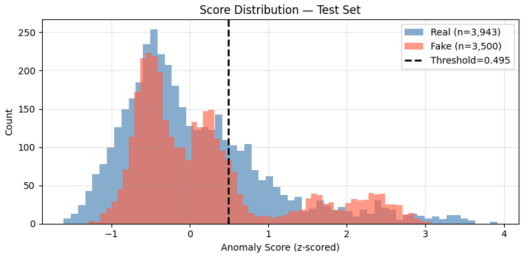

Figure 14: Test score distribution.

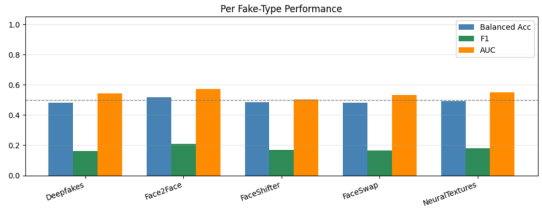

Figure 15: Per fake-type performance.

Per-fake-type evaluation also showed limited separation:

Fake Type	Balanced Accuracy	F1	AUC
Deepfakes	0.4802	0.1621	0.5436
Face2Face	0.5159	0.2104	0.5727
FaceShifter	0.4852	0.1690	0.5037
FaceSwap	0.4824	0.1650	0.5328
NeuralTextures	0.4931	0.1798	0.5486

Table 11: fake-type.

Among the tested fake types, Face2Face achieved the strongest separation, while FaceShifter produced the weakest results. Overall, the AUC values remained relatively low across all categories.

6.2.6 Challenges Encountered

Challenge 1: Weak real/fake separation. Although reconstruction learning was stable, the anomaly score did not provide strong separation between real and fake samples.

Challenge 2: Score overlap. The score distributions of real and fake images overlapped significantly, which made threshold selection sensitive and reduced classification reliability.

Challenge 3: Sensitivity to model design. Several intermediate variants were tested, including stronger latent constraints, perceptual loss, and alternative regularization terms. Some of these variants improved reconstruction but reduced detection performance or produced unstable score behavior.

Challenge 4: Limitation of pure anomaly detection. The experiments suggest that, on this dataset, a pure autoencoder-based anomaly-detection method is limited in its ability to capture subtle deepfake artifacts consistently across all fake types.

6.2.7 Solutions and Next Steps

To improve the stability of this solution, the final model was simplified by removing unstable components such as perceptual loss, strong latent constraints, and complex score-shaping terms. The final version keeps only the most reliable elements: compact architecture, light preprocessing, aligned anomaly scoring, and validation-based threshold selection.

Potential next steps:

1. Use patch-level anomaly analysis to capture local manipulation artifacts more effectively.
2. Focus on important facial regions such as the eyes, mouth, and blending boundaries.
3. Improve the anomaly score by selecting the most useful scoring components.
4. Increase input resolution to preserve subtle deepfake artifacts.
5. Analyze reconstruction residual maps to better localize manipulated regions.

6.3 Solution #3: Hybrid Feature Fusion with SVM - [Mohammed Ali Alqurayqiri]

6.3.1 Approach and Model Selection

A classical machine learning pipeline fusing spatial, frequency, and texture features classified by an RBF SVM, achieving 66.48% test accuracy on FF++ C23 and 65.73% on the expanded master split - establishing a principled baseline against the deep learning solution (Section 6.1).

Face manipulations leave traces in three complementary domains. Spatial features capture structural inconsistencies via a frozen ResNet50 (2048-dim). Frequency features expose GAN spectral artifacts via DCT on chroma channels. Texture features detect unnatural skin smoothness via LBP histograms. Fusing all three gave +6.64% over ResNet50 alone (59.84% $\rightarrow$ 66.48%).

The SVM was chosen for its strong performance in high-dimensional, limited-sample settings. A LinearSVC diagnostic confirmed the feature space is not linearly separable (59.54% test accuracy, 5.07 pt gap), making the RBF kernel necessary.

6.3.2 Model Architecture/Configuration

Three parallel extractors process each frame, aggregate per video, then fuse into a single vector for classification.

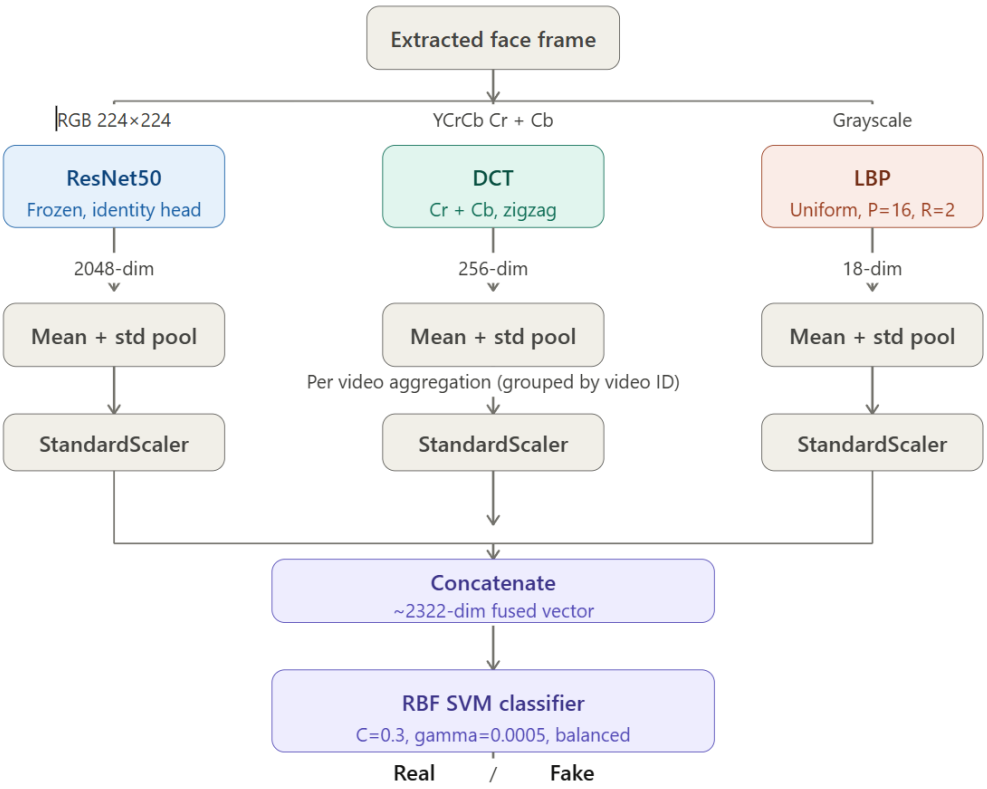

Figure 16: Three parallel extractors process each frame, aggregate per video, then fuse into a single vector.

Modality	Extractor	Input	Dimensionality
Spatial	Frozen ResNet50 (ImageNet)	RGB 224x224	2048-dim
Frequency	DCT on Cr + Cb chroma channels	YCrCb	128 + 128 = 256-dim
Texture	Uniform LBP (P=16, R=2)	Grayscale	18-dim
Fused	Concatenated after independent scaling		**2322-dim**

Video-level aggregation: Frames are pooled per video using mean + standard deviation, producing one vector per video and preventing frame-signature memorisation.

Scaling: ResNet50 and handcrafted blocks are scaled independently with StandardScaler before concatenation, ensuring neither block dominates the SVM's distance calculations.

Classifier: RBF SVM with C=0.3, gamma=0.0005, balanced class weights. Cross-validation uses GroupKFold(n_splits=3) grouped by video ID.

Regularisation study: PCA retaining 452 components (95.0% variance) was tested but showed a consistent tradeoff - every gap reduction cost proportional test accuracy, so the no-PCA configuration was selected as the final model.

6.3.3 Implementation Details

Libraries: PyTorch (ResNet50 GPU extraction), scikit-learn (SVC, StandardScaler, GroupKFold, PCA), OpenCV (DCT, YCrCb), scikit-image (LBP), NumPy, Matplotlib/Seaborn.

Environment: Kaggle notebook, Tesla T4 GPU. ResNet50 runs on GPU; SVM on CPU.

Key note: StandardScaler must be fit only on the training fold of each GroupKFold split - fitting on the full dataset before splitting introduces leakage, a bug caught early after observing inflated training accuracy.

6.3.4 Current Implementation Status

Component	Status
Data loading and preprocessing	Complete
Feature extraction (ResNet50, DCT, LBP)	Complete
Video-level aggregation and fusion	Complete
SVM training and hyperparameter tuning	Complete
Regularisation study (PCA, LinearSVC)	Complete
Evaluation metrics and plots	Complete
Expanded dataset experiments	Complete
Unified pipeline on new dataset	In progress
Cross-solution final comparison	In progress
Final report write-up	In progress

6.3.5 Preliminary Results

Original Dataset (FF++ C23)

#	Configuration	CV Acc	Val Acc	Test Acc	Train/Test Gap
1a	DCT + LBP only	57.76%	58.86%	57.40%	13.61 pts
1b	ResNet50 only	60.78%	61.02%	59.84%	13.86 pts
2	Full fusion, no PCA	65.37%	66.43%	66.48%	22.83 pts
3	Full fusion + PCA (452)	60.82%	60.25%	59.66%	13.49 pts
4	LinearSVC + PCA (452)	60.34%	60.70%	59.54%	5.07 pts

Table 12: Experiment progression.

The +6.64% gain from fusion confirms DCT and LBP contribute complementary signal. The $\sim$23 pt train/test gap is structural - the LinearSVC result (5.07 pt gap, 59.54% accuracy) proves it reflects genuine non-linearity, not RBF overfitting. The root cause is frozen ImageNet weights encoding video-specific characteristics that do not transfer to unseen videos, compounded by H.264 compression artifacts in C23.

Expanded Dataset (Master Split)

Class	Precision	Recall	F1	Support
Fake	0.86	0.65	0.74	30,627
Real	0.39	0.68	0.49	9,900
Weighted avg	0.75	0.66	0.68	40,527
Test Accuracy	**65.73%**			

Table 13: Expanded dataset classification report (Test set).

Best params: C=0.3, gamma=1e-05. PCA: 452 components, 93.6% explained variance.

Metric	Original Dataset	Expanded Dataset
Test Accuracy	66.48%	65.73%
Train Accuracy	89.31%	67.29%
Train/Test Gap	22.83 pts	**∼1.56 pts**
PCA components	452 (95.0% var)	452 (93.6% var)
Test samples	8,944	40,527

Table 14: Original vs Expanded dataset comparison.

The most significant finding is the gap reduction from 22.83 to ∼1.56 pts, indicating a substantial improvement in overfitting. The larger, more diverse dataset forces the model to learn general manipulation indicators rather than video-specific signatures, while test accuracy remained stable at 65.73%.

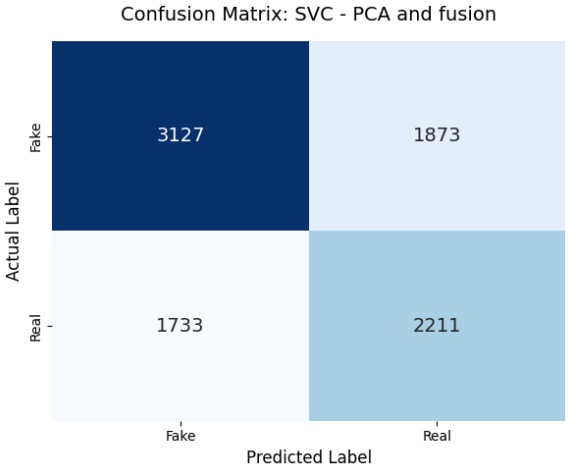

Figure 17: Confusion Matrix (Expanded Dataset).

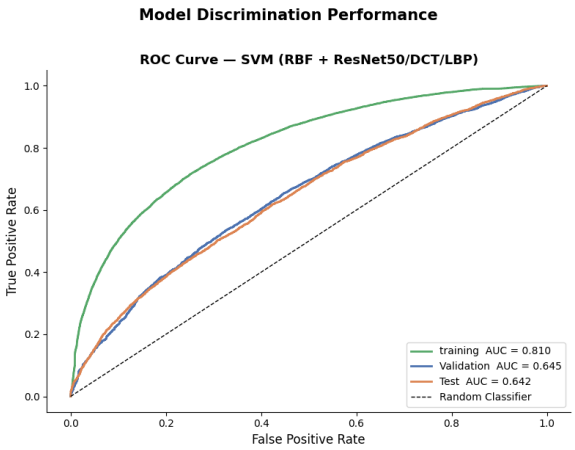

Figure 18: ROC Curve (Expanded Dataset).

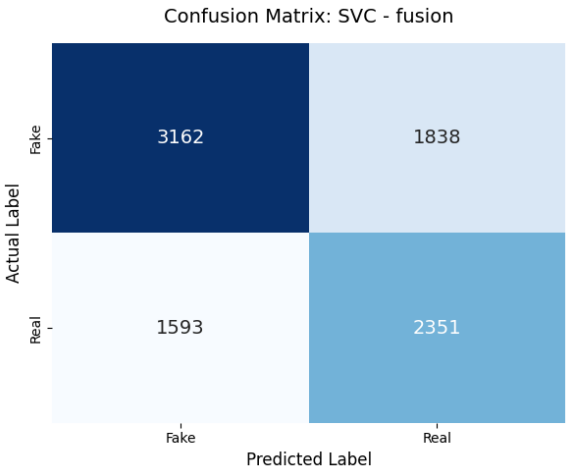

Figure 19: Confusion Matrix (Original FF++).

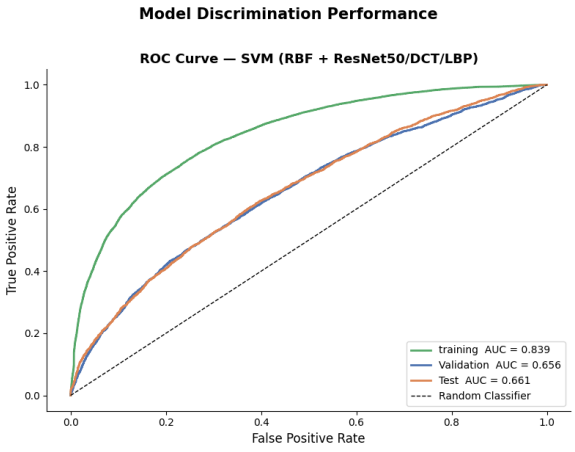

Figure 20: ROC Curve (Original FF++).

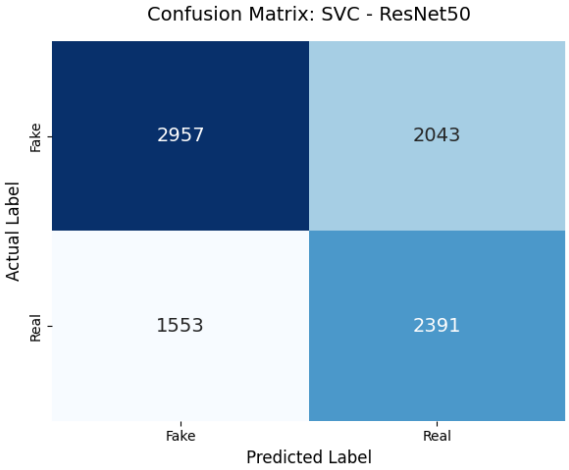

Figure 21: Confusion Matrix (ResNet50 only).

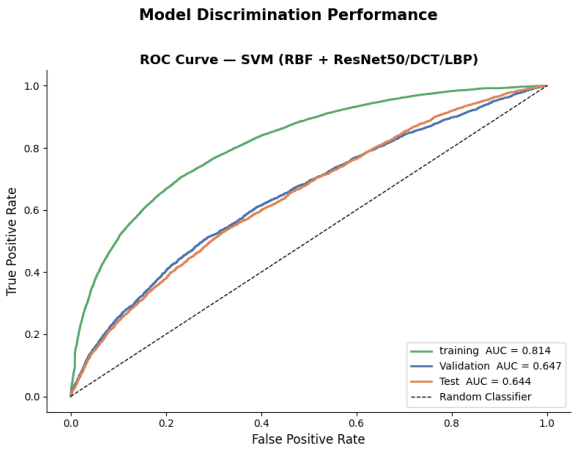

Figure 22: ROC Curve (ResNet50 only).

6.3.6 Challenges Encountered

Challenge 1 - Data leakage from frame-level splitting (Critical): A naive random split placed frames from the same video into both training and validation simultaneously, causing the model to memorise identities rather than detect manipulation. Fixed by switching to GroupKFold partitioned by video ID.

Challenge 2 - Scaler leakage: Fitting StandardScaler on the full dataset before splitting introduced a statistical dependency across splits. Fixed by refitting the scaler inside each fold only.

Challenge 3 - Persistent train/test gap: The 2322-dim fused vector is large relative to ∼600-800 video-level training samples. No regularisation strategy closed the gap without proportionally sacrificing test accuracy, pointing to a ceiling in the frozen feature representations.

Challenge 4 - Hyperparameter tuning instability: Early grid searches used a single held-out validation set, introducing selection bias. Switching to GroupKFold grid search produced stable selections (C=0.3, gamma=0.0005).

6.3.7 Solutions and Next Steps

Challenge	Solution	Result
Frame-level leakage	GroupKFold by video ID	Trustworthy evaluation
Scaler leakage	Scaler fit inside each fold only	Honest validation accuracy
Train/test gap	Full regularisation study (PCA, LinearSVC)	Gap documented and explained
Tuning instability	GroupKFold grid search	Stable hyperparameters
Overfitting	Expanded master split	Gap reduced from 22.83 to ∼1.56 pts

Remaining work:

1. Unified pipeline on new dataset: Finalise evaluation on the expanded master split under the team's shared setup.
2. Cross-solution comparison: Collaborate with the deep learning teammate for a side-by-side comparison on accuracy, recall balance, and compute cost.

3. Evaluation plots: Insert confusion matrix figures and ROC curves at the marked placeholders once generated from the final test set.

4. Report consolidation: Integrate all remaining results and plots into the final submission.

6.4 Solution #4: Mesoscopic Feature Extraction via MesoInception-4 - [Rakan Fawuzi Jalalah]

6.4.1 Approach and Model Selection

Our proposed solution for detecting manipulated face videos is a deep convolutional neural network based on the MesoInception-4 architecture. Rather than relying on traditional macroscopic analysis (which evaluates the entire face structurally) or microscopic analysis (which focuses on invisible pixel-level image noise), this solution is specifically designed to target mesoscopic properties.

These are intermediate-level visual artifacts, such as unnatural blending boundaries, subtle texture degradation, and localized compression inconsistencies inherently introduced by generative models and face-swapping algorithms.

This specific approach was chosen for several critical reasons:

1. **Targeted Artifact Detection:** Deepfake generation techniques have evolved to produce highly realistic full faces that easily fool macroscopic classifiers. Furthermore, social media compression often destroys microscopic noise patterns. The mesoscopic approach specifically targets the intermediate ”sweet spot” where manipulation flaws remain most visible and resilient to compression.

2. **Multi-scale Feature Extraction:** MesoInception-4 replaces standard convolutional layers with Inception modules. By employing parallel convolutional filters of varying sizes (1x1, 3x3, and 5x5), the model can simultaneously examine the input image at multiple spatial resolutions, allowing it to capture manipulation artifacts of varying sizes without losing local context.

3. **Computational Efficiency:** Compared to massive, parameter-heavy architectures (e.g., ConvNeXt Large or Vision Transformers), MesoInception-4 offers a highly lightweight and efficient topology. This ensures an optimal trade-off, providing high detection accuracy and an 87.21% recall rate while maintaining low inference latency and minimal GPU memory requirements, making it highly practical for real-world, scalable deployment.

6.4.2 Model Architecture/Configuration

The proposed solution utilizes the MesoInception-4 architecture, tailored for high-resolution input and efficient mesoscopic feature extraction. The network is configured to ingest RGB images resized to 384x384x3, a deliberate hyperparameter choice that preserves subtle blending boundaries better than the standard 256x256 input size.

1. **Inception Modules (Multi-scale Feature Extraction):** Unlike sequential CNNs, the first two stages of the network consist of Inception modules. To capture deepfake artifacts occurring at various structural scales (e.g., eye distortions vs. full-mask blending), each Inception module applies parallel convolutional filters of different spatial dimensions: 1x1, 3x3, and 5x5. The feature maps from these parallel convolutions are then concatenated. This multi-scale approach allows the network to learn both localized pixel-level anomalies and broader spatial inconsistencies simultaneously. All intermediate convolutional layers utilize the ReLU (Rectified Linear Unit) activation function to introduce non-linearity and mitigate the vanishing gradient problem.

2. **Standard Convolutional and Pooling Layers:** Following the two Inception modules, the architecture features two standard convolutional blocks. Each block is equipped with 5x5 filters and is followed by a Max-Pooling (2x2) layer. These pooling layers are strictly configured to down-sample the spatial

dimensions of the feature maps, drastically reducing the computational complexity while retaining the most prominent mesoscopic artifacts.

3. **Fully Connected and Classification Layers:** The output from the final pooling layer is flattened into a 1D vector and passed through a highly regularized dense neural network. A hidden Dense layer (typically 16 units) is applied, followed by a Dropout layer (p=0.5) to inject stochastic noise during training, heavily reducing the risk of overfitting. The final classification is performed by a single-neuron Dense layer utilizing the Sigmoid activation function, which maps the network's output to a continuous probability distribution between 0 (Fake) and 1 (Real).

4. **Hyperparameters and Training Configuration:**
   - Input Dimension: 384x384x3
   - Batch Size: 16 (optimized for GPU memory constraints)
   - Loss Function: Binary Crossentropy
   - Optimization: The network was trained using Mixed Precision (FP16/FP32) to accelerate tensor operations and maximize GPU utilization without sacrificing gradient stability.

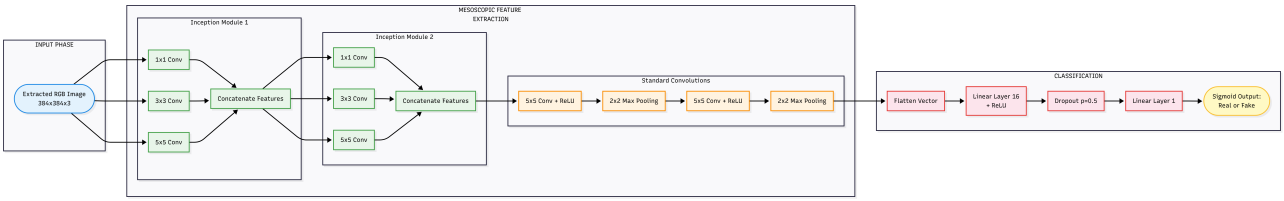

Figure 23: MesoInception-4 multi-scale architecture.

Hyperparameter	**Training Configuration**
Training Strategy	End-to-end Training
Max Epochs	50 (Controlled by Early Stopping)
Optimizer	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Initial LR	1e-3
LR Scheduler	ReduceLROnPlateau (factor=0.5, patience=3)
Loss Function	Binary Crossentropy
Batch / Image	16 / 384x384 RGB
Precision	Mixed Precision (FP16/FP32)
Regularization	Dropout (p=0.5), Early stopping (patience=5)

Table 15: Training Hyperparameters.

Parameter	**MesoInception-4 Configuration**
Input Dimension	384x384x3 RGB Image
Feature Extraction	Stacked Inception Modules (parallel 1x1, 3x3, 5x5 filters)
Down-sampling	Standard Convolutions (5x5) + Max-Pooling (2x2)
Classification Head	Dense Layer (16 units) $\rightarrow$ Dropout (p=0.5) $\rightarrow$ Sigmoid
Optimizer & Loss	Mixed Precision (FP16/FP32), Binary Crossentropy
Model Footprint	Highly Lightweight (Optimized for fast inference)

Table 16: Architectural and Hyperparameter Configuration for the MesoInception-4 Model.

6.4.3 Implementation Details

Libraries: TensorFlow 2.x (training, Keras API, Mixed Precision), NumPy (algorithmic dataset balancing), OpenCV (high-resolution image loading and resizing to 384x384), scikit-learn (evaluation metrics and ROC-AUC computation).

Algorithmic Data Balancing: To prevent the ”accuracy paradox” caused by severely skewed real-world data, a programmatic down-sampling algorithm was deployed. The majority class was automatically truncated to match the minority limit, yielding a strict 50/50 balanced evaluation subset (19,800 images: 9,900 Real / 9,900 Fake) with zero prediction bias.

Training Execution & Callbacks: The MesoInception-4 network was trained end-to-end. Dynamic Keras callbacks were integrated, specifically ReduceLROnPlateau to dynamically halve the learning rate upon validation loss stagnation, and EarlyStopping to halt training and prevent overfitting.

Dataset (Evaluation Split): The initial dataset was processed into standard training, validation, and testing sets. While the training phase utilized the heavily imbalanced data to maximize feature exposure, the final testing split was strictly regulated to ensure fair metric computation.

6.4.4 Current Implementation Status

- Data loading and preprocessing: Complete (Strictly balanced 19,800 evaluation images for unbiased testing).
- Model training: In Progress (Baseline MesoInception-4 model successfully trained; currently experimenting with deeper architectural variations).
- Evaluation: Complete for baseline (Initial 80.27% accuracy achieved; further evaluation planned for optimized versions).
- Hyperparameter tuning: In Progress (Ongoing optimization of learning rates and dropout ratios to maximize generalization).

6.4.5 Preliminary Results

This section presents the preliminary empirical evidence derived from the evaluation of the baseline MesoInception-4 model. The focus is placed on visualizing the critical impact of logical data balancing on the model's performance. All metrics and matrices demonstrate a clear transition from majority-class bias to strictly balanced discrimination.

Evaluation Metric	Baseline Result (Value)
Test Accuracy (General)	77.09%
Balanced Test Accuracy	80.27%
Test Loss (Log-Loss)	0.4357
Precision	76.45%
Recall (Authentic Class)	87.21%
F1-Score	81.48%
Area Under Curve (AUC)	0.8917

Table 17: Comprehensive Preliminary Evaluation Metrics (Post-Balancing).

1. Pre-Balancing Analysis (Imbalanced Raw Test Set)

The initial testing was conducted on the heavily skewed raw test set, where manipulated images dominated the authentic samples.

Class	Samples	Dist. (%)
Authentic (Real)	9,900	23.5%
Manipulated (Fake)	32,228	76.5%

Table 18: Input Distribution (Raw Unbalanced Hold-out Test Split).

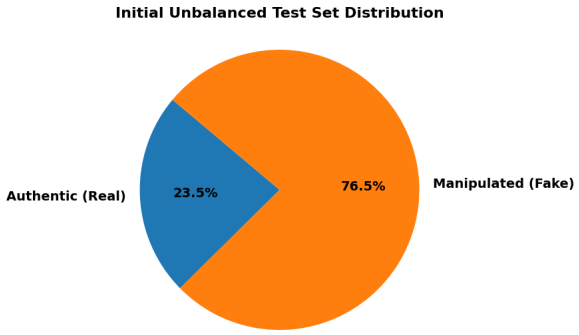

Figure 24: Initial unbalanced test set distribution.

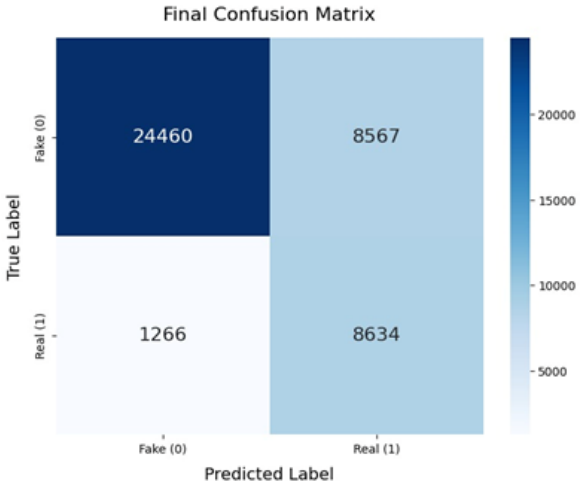

Figure 25: Performance baseline on unbalanced testing data.

2. Post-Balancing Analysis (Balanced Evaluation Subset)

To ensure fair performance metric extraction, the programmatic down-sampling algorithm (The ”Bulldozer”) was applied to strictly truncate the majority class.

Class	Samples	Dist. (%)
Authentic (Real)	9,900	50.0%
Manipulated (Fake)	9,900	50.0%

Table 19: Input Distribution (The Algorithmically Balanced Subset).

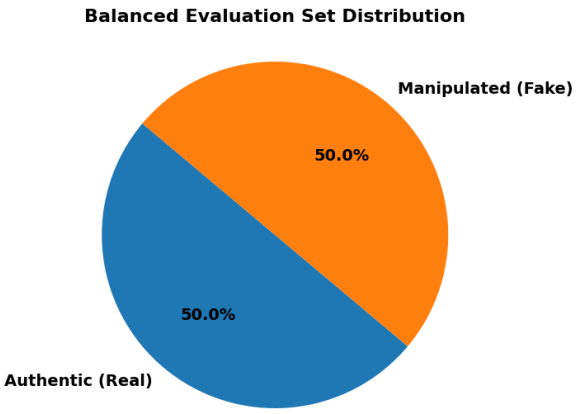

Figure 26: Strictly balanced evaluation set distribution.

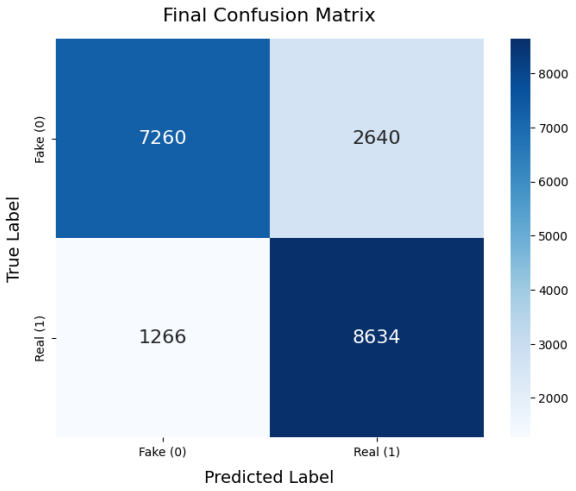

Figure 27: Final balanced confusion matrix.

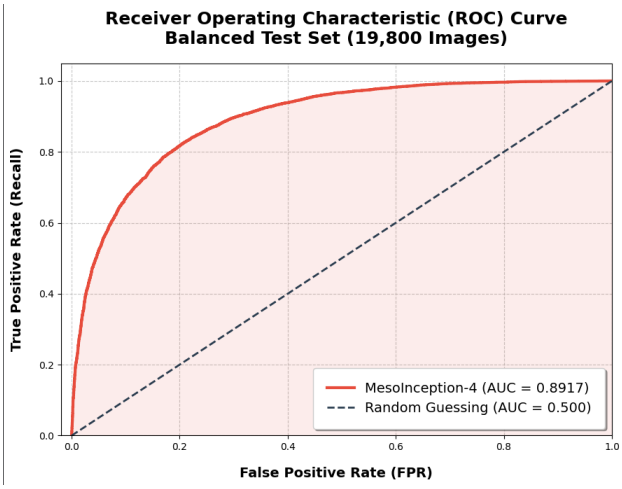

Figure 28: Receiver Operating Characteristic (ROC).

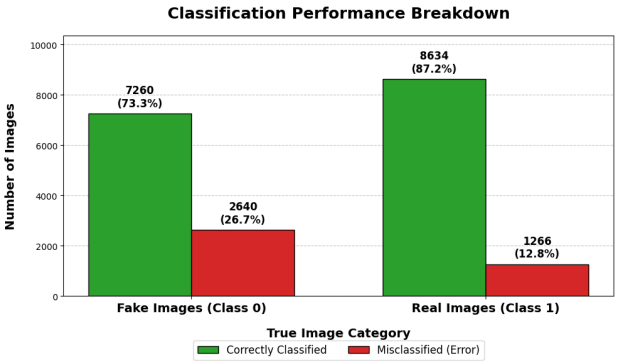

Figure 29: Detailed classification performance breakdown.

Category	Total Images	Accuracy (%)
Real Images	7,800	87.12%
Fake Celeb-synthesis	4,069	92.38%
Fake: DeepFake	725	77.10%
Fake: Face2Face	734	73.30%
Fake: FaceShifter	770	71.69%
Fake: FaceSwap	747	60.00%
Fake: Neural Textures	755	43.18%

Table 20: Per-Category Testing Accuracy Breakdown.

3. Optimization Timeline (In Progress)

While the preliminary balanced accuracy of 80.27% is promising, ongoing architectural optimizations are focused on deepening the network's feature extraction layers.

6.4.6 Challenges Encountered

- **Data Imbalance (Data Problem):** The initial dataset was heavily skewed, with manipulated images making up about 76% of the total samples. This imbalance caused the model to biasedly predict the majority class to achieve ”easy” accuracy. To solve this, I developed a custom Python script for algorithmic down-sampling, which strictly truncated the majority class to create a perfectly balanced 50/50 evaluation set.
- **Data Leakage Risks (Conceptual Challenge):** I identified a risk where frames from the same video source could appear in both training and testing sets, leading to ”Data Leakage.” This would make the model memorize specific backgrounds instead of learning deepfake features. I manually restructured the data pipeline to ensure that each unique video ID is strictly isolated to a single split (Training or Testing).
- **VRAM & GPU Limitations (Computational Limitation):** Training the MesoInception-4 model on high-resolution 384x384 images frequently caused Google Colab to crash with ”Out of Memory” (OOM) errors. I resolved this by implementing Mixed Precision training and carefully reducing the batch size to 16, finding the optimal balance between training stability and memory usage.
- **Overfitting on Deep Architectures (Technical Issue):** Due to the depth of the model, it initially showed signs of overfitting, performing well on training data but poorly on validation. I addressed this by fine-tuning the Dropout layers (p=0.5) and integrating an EarlyStopping callback, which I monitored closely to halt training at the most generalized state.

6.4.7 Solutions and Next Steps

How Challenges are being Addressed:

- **Automated Balancing Pipeline:** To maintain a fair evaluation, the custom down-sampling script is now a permanent part of the preprocessing pipeline. Every new test set is automatically balanced to 50/50 before evaluation to prevent any class bias.
- **Optimized Resource Management:** To handle the GPU memory limitations, I am utilizing Mixed Precision training and monitoring system resources in real-time. This allows the model to train on larger images without crashing the environment.
- **Validation-Driven Training:** To stop overfitting before it starts, I am using a combination of EarlyStopping and manual checkpointing. I save the model only when it achieves the best validation loss, ensuring that the version we use is the most generalized one.

Next Steps for Final Completion:

- **Architectural Deepening (Length Improvement):** My next primary step is to evolve the baseline MesoInception-4 by adding extra feature extraction layers. This ”deeper” version aims to bridge the gap between general and balanced accuracy.
- **Hyperparameter Fine-Tuning:** I will conduct an extensive grid search for the optimal learning rate and dropout ratio for the new deeper architecture to maximize the F1-score.
- **Extended Comparative Analysis:** Before the final deadline, I will compare the current baseline results with the improved version to document the performance gain.

7 Preliminary Comparative Analysis

7.1 Current Results Summary

Solution Name	Status	Accuracy	Precision	Recall	F1-Score
SOA (v10ab)	Complete	**93.74%**	*In Progress*	*In Progress*	*In Progress*
Sol 1: ConvNeXt V2 + FFT	Complete	**97.39%**	97.00%	97.00%	97.40%
Sol 2: Autoencoder	In Progress	50.65%	45.27%	23.66%	31.08%
Sol 3: Hybrid SVM Fusion	Complete	65.73%	75.00%	66.00%	68.00%
Sol 4: MesoInception-4	In Progress	80.27%	76.45%	87.21%	81.48%

Table 21: Current Metric Summary across all parallel evaluation pathways.

7.2 Early Observations & Trade-offs

Initial empirical evaluations revealed distinct disparities in the network's ability to classify different manipulation typologies. Macroscopic spatial manipulations yield accuracy rates exceeding 95%, whereas micro-manipulations (e.g., NeuralTextures) result in precipitous detection drops across all models. We anticipate a significant trade-off between the global semantic understanding of the massive Transformer/CNN architectures and the lightweight inference speed of the MesoInception-4 and classical SVM baselines.

8 Project Challenges and Risk Mitigation

8.1 Technical Challenges & Mitigation

- **Compiler Constraints & VRAM Allocation:** Dynamic resizing and blending of tensors during Asymmetric Mixup interacted adversely with the PyTorch compiler (`torch.compile`), causing rapid memory leaks and massive Out-of-Memory (OOM) spikes exceeding our 102GB limits.
- **Solution:** `torch.compile` was globally disabled to mathematically guarantee the stability of gradient allocations during training.

8.2 Timeline and Resource Challenges

Extracting and processing 360,000 discrete frames required significantly more network bandwidth and storage I/O than originally provisioned, creating brief pipeline bottlenecks.

9 Revised Timeline to Final Submission

- **Week of April 20, 2026:** Finalize hyperparameter matrices for the v10ab architecture and deeper MesoInception variations.
- **Week of April 25, 2026:** Complete final metrics generation across all cross-solution testing sets.
- **Week of April 30, 2026:** Institute a total code freeze; initiate unified comparative inference analysis across all solutions.
- **Week of May 3, 2026:** Draft final technical manuscript; execute theoretical analysis of latency versus accuracy trade-offs.
- **May 10, 2026:** Execute final presentation and submit deliverables.

10 References

1. Yermakov, A., Cech, J., Matas, J., & Fritz, M. (2025). *Deepfake Detection that Generalizes Across Benchmarks*. arXiv:2508.06248v3.
2. Yang, Y., Qian, Z., Zhu, Y., Russakovsky, O., & Wu, Y. (2024). *Scaling Up Deepfake Detection by Learning from Discrepancy*. arXiv:2404.04584v2.
3. Haliassos, A., et al. (2021). *Lips Don't Lie: A Generalisable and Robust Approach To Face Forgery Detection*. CVPR 2021.
4. Shiohara, K., & Yamasaki, T. (2022). *Detecting Deepfakes With Self-Blended Images*. CVPR 2022.
5. Cui, Y., et al. (2024). *Forensics Adapter: Unleashing CLIP for Generalizable Face Forgery Detection*.
6. Yan, Z., et al. (2024). *Orthogonal Subspace Decomposition for Generalizable AI-Generated Image Detection*.
7. UNESCO. *Deepfakes and the crisis of knowing*.
8. The World Economic Forum. *Detecting dangerous AI is essential in the deepfake era*.
9. *Seeing Isn't Believing: Addressing the Societal Impact of Deepfakes in Low-Tech Environments*. arXiv:2508.16618v1.
10. Adaptive Security. *Deepfake Protection Guide: 9-Step Risk Framework for 2026*.